Glava 4

Keš memorija i lažno deljenje

Cilj vežbe

- Povezati redosled adresa, linije keša i ponovnu upotrebu podataka.
- Realizovati dva prolaza kroz matricu i proveriti isti rezultat.
- Uporediti rasporede brojača uz isti rad i istu atomsku operaciju.
- Objasniti merenje pomoću radnog skupa i vlasništva nad linijom keša.

U prethodnim vežbama niti su delile ulaz i pisale različite izlaze pokretne srednje vrednosti. Sada ispitujemo kako redosled pristupa i položaj podataka u memoriji utiču na vreme. Za lokalnost koristimo dva prolaza kroz istu matricu, a za lažno deljenje nezavisne brojače niti. Sve komande pokretati iz `root_dir`.

Fajlovi za ovu vežbu nalaze se u arhivi `lab_04.tar.gz`. Izvorni kod je u `src/arch/`. Dopunjujete samo dve funkcije u `cache_kernels.cpp`. Priloženi su programi za proveru i merenje, kao i skripte za crtanje grafika.

4.1 Predviđanje adresa i radnog skupa

Memoriju posmatramo kao niz bajtova, od kojih svaki ima svoju adresu. Elementi matrice smešteni su po redovima u jednodimenzionalnom nizu: najpre ceo red 0, zatim red 1 i tako dalje. Koristimo sledeće oznake:

Oznaka	Značenje
N	Broj redova i broj kolona kvadratne matrice; ukupno ima N^2 elemenata.
i, j	Indeksi reda i kolone. Oba počinju od 0 i uzimaju vrednosti do $N-1$.
b	Veličina jednog elementa u bajtovima, odnosno `sizeof(int)` za ovaj program.
A_0	Adresa početka matrice: adresa prvog bajta elementa `a[0]`.

Pre elementa u redu i i koloni j nalazi se $iN + j$ elemenata: i celih redova sa po N elemenata, pa još j elemenata tekućeg reda. Množenjem tog broja sa b dobijamo pomeraj u bajtovima od početka matrice. Oznakom $\mathrm{addr}(i, j)$ predstavljamo adresu prvog bajta izabranog elementa, pa važi

$$\mathrm{addr}(i,j) = \underbrace{A_0}_{\text{početna adresa}} + \underbrace{(iN+j)b}_{\text{pomeraj u bajtovima}} .$$

Prolaz po redu menja j, sa korakom od b bajtova. Prolaz po koloni menja i, sa korakom od Nb bajtova. Jedna linija keša obuhvata L susednih bajtova; veličina linije L takođe je izražena u

bajtovima. Prostorna lokalnost znači da se ubrzo koriste bliske adrese; vremenska lokalnost znači ponovnu upotrebu istih podataka.

Za model sa $N = 8$, $b = 4$ B i $L = 16$ B, uzmimo početnu adresu $A_0 = 4096$. Za element u redu 1 i koloni 2 dobijamo

$$\mathrm{addr}(1, 2) = 4096 + (1 \cdot 8 + 2) \cdot 4 = 4136.$$

Njegov pomeraj od početka matrice je 40 B. Za prva četiri elementa reda 0 pomeraji su 0, 4, 8 i 12 B; za prva četiri elementa kolone 0 pomeraji su 0, 32, 64 i 96 B.

Početna adresa 4096 deljiva je sa 16, pa se početak matrice poklapa sa početkom jedne linije keša. Tu liniju u ovom modelu označavamo brojem 0. Ona pokriva adrese 4096–4111, naredna linija 4112–4127 i tako dalje. Relativni broj linije dobijamo kao

$$\left\lfloor \frac{\mathrm{addr}(i, j) - A_0}{L} \right\rfloor .$$

Za adresu 4136 dobijamo $\lfloor 40/16 \rfloor = 2$, odnosno treću liniju od početka matrice. Ovo numerisanje važi za navedeni model u kojem matrica počinje na granici linije keša.

1. Zapisati prvih osam pomeraja za red 0 i kolonu 0. Koliko različitih linija dodiruje svaki niz od osam pristupa?

2. Kada sledeća kolona ponovo koristi liniju prve kolone? Šta može izbaciti tu liniju pre ponovne upotrebe?

3. Da li broj različitih linija sam određuje broj stvarnih promašaja? Razmotriti početno stanje keša i dovlačenje podataka unapred.

Ukupnu veličinu podataka matrice označavamo sa $D = N^2 b$, u bajtovima. Pre merenja izračunati D za $N = 128, 512, 2048$ i uporediti sa keševima na radnom računaru. Za $b = 4$ B to su 64 KiB, 1 MiB i 16 MiB; 1 KiB = 1024 B, a 1 MiB = 1024^2 B. Program ispisuje stvarno `sizeof(int)`. Predvideti za koju veličinu će razlika redosleda biti najveća. Kapacitet sam nije dovoljan: više linija može konkurisati za ista mesta u kešu.

4.2 Zadatak: dva prolaza kroz istu matricu

U zaglavlju `cache_kernels.hpp` su dogovoreni tipovi i funkcije:

```
using Matrix = vector<int>;
using Sum = long long;

Sum sum_rows(const Matrix& a, int n);
Sum sum_columns(const Matrix& a, int n);
```

U ovom primeru naziv `Matrix` označava niz celih brojeva, a `Sum` je kraće ime za širi tip sume. Za indekse u zadatim granicama dovoljan je tip `int`. Za sumu koristimo tip `long long`, koji sprečava prekoračenje pri sabiranju većeg niza.

Dopuniti funkcije u izvornom fajlu `cache_kernels.cpp`:

1. U `sum_rows` spoljašnja petlja bira red i, a unutrašnja kolonu j. Element se čita kao `a[i*n+j]`.

2. U `sum_columns` zameniti redosled petlji: spoljašnja bira kolonu j, a unutrašnja red i. Adresni izraz ostaje isti.

3. U obe funkcije početi lokalnom sumom nula, posetiti svaki element jednom i vratiti sumu. Ulaz se samo čita. U funkcijama ne alocirati, ne ispisivati i ne meriti vreme.

Polazni kod prijavljuje nezavršen zadatak dok obe funkcije nisu dopunjene. Program implementiran u izvornom fajlu `locality.cpp` prihvata $1 \le N \le 8192$, proverava argumente i priprema memoriju pre poziva funkcija. Veću matricu birati samo ako staje u raspoloživi RAM.

4.2.1 Provera ručnim računanjem

Pre pokretanja programa ručno izračunati zbir elemenata za matricu sa jednim elementom $[-3]$, matricu 2×2 sa elementima $[1, 2, 3, 4]$ i sledeću matricu:

$$\begin{bmatrix} 4 & -2 & 7 \\ 1 & -5 & 3 \\ 0 & 8 & -6 \end{bmatrix}.$$

Za prvu matricu očekivani zbir je -3, a za preostale dve je 10. Ugrađena provera upoređuje rezultate obe funkcije sa tim vrednostima. Za dodatnu proveru upotrebiti matricu

$$\begin{bmatrix} -5 & 0 \\ 7 & -1 \end{bmatrix}$$

sa očekivanim zbirom 1, ili osmisliti svoj primer. U oba slučaja prvo ručno izračunati zbir, pa u funkciju `small_checks` dodati matricu i očekivani rezultat.

```
g++ -O3 -Wall -Wextra src/arch/locality.cpp \
    src/arch/cache_kernels.cpp -o src/locality
./src/locality --check
./src/locality 8 row-first locality_8.csv
./src/locality 0
./src/locality 8abc
```

Poslednja dva poziva treba da prijave grešku. Provera mora proći pre tumačenja vremena. Za merenu matricu program `locality` postavlja `a[i] = i % 17 - 8` i istovremeno računa referentnu sumu. Posle svakog prolaza poredi rezultat sa njom.

4.2.2 Šta ulazi u vreme

Merenje koristi `steady_clock`, kao u prvoj vežbi. Između dva čitanja časovnika je samo poziv funkcije za sabiranje. Alokacija, popunjavanje, provere i ispis su izvan intervala. Rezultat svakog poziva koristi se posle merenja; akumulator je obična lokalna promenljiva. Kompajler može da vektorizuje petlju ili promeni drugi detalj izvršavanja. Porediti varijante sa istim kompajlerom i opcijama; optimizacija je deo eksperimenta.

Program `locality` najpre jednom izvrši svaku varijantu, bez čuvanja izmerenog vremena. Zatim za svaku varijantu zabeleži vremena pet narednih izvršavanja. Na neparnom ponavljanju prvo izvršava izabrani početni prolaz, a na parnom drugi. Argument `column-first` menja početni izbor:

```
./src/locality 512 row-first locality_512.csv
./src/locality 512 column-first locality_512_reverse.csv
```

Oba prolaza su u svakom pozivu. Pripremno izvršavanje, popunjavanje matrice i prethodni prolaz utiču na stanje keša; promena redosleda ne prazni keš. U CSV fajlu čuvaju se varijanta, N, ponavljanje, mesto u redosledu, vreme i suma. Ako se ime CSV-a izostavi, koristi se `locality.csv`; za novu seriju birati novo ime da se stara vremena sačuvaju.

4.3 Lažno deljenje: različiti brojači u jednoj liniji

Ako dve niti često menjaju različite promenljive u istoj liniji keša, pravo na upis cele linije može prelaziti između jezgara. To je lažno deljenje. Kod stvarnog deljenja menjaju istu promenljivu. Čitanje istog nepromenljivog ulaza, kao kod prozora pokretne srednje vrednosti, može koristiti keširane kopije bez takvih prelaza prava na upis.

Priloženi program za poređenje rasporeda brojača implementiran je u izvornom fajlu `false_sharing.cpp`. Za tip brojača koristi se kraće ime `Count` za širi neoznačeni celobrojni

tip. U trećoj vežbi atomsko ažuriranje uvedeno je direktivom `#pragma omp atomic`. U C++ programu istu ideju primenjujemo pomoću šablona `atomic<Count>` iz zaglavlja `<atomic>`.

Atomsko ažuriranje je nedeljivo čitanje i izmena jednog objekta. Ako dve niti istovremeno dodaju po jedan istom atomskom brojaču, oba uvećanja ostaju sačuvana. Poziv `value.fetch_add(1)` atomski uvećava brojač za jedan i vraća njegovu prethodnu vrednost. Na primer, za početnu vrednost 7 vraća 7, a u brojaču ostavlja 8. U prikazanom programu povratna vrednost se zanemaruje. Za atomsko čitanje trenutne vrednosti služi metod `load()`.

Oba rasporeda koriste istu operaciju sa podrazumevanim podešavanjima. Pomoću metoda `is_lock_free()` proveravamo da li se atomske operacije nad objektom realizuju bez unutrašnjeg zaključavanja biblioteke; vrednost 1 označava takav slučaj. Za izmenu brojača i dalje je potrebno pravo na upis odgovarajuće linije keša.

4.3.1 Prvo funkcija niti, zatim njeno pokretanje

```
void increment(atomic<Count>& value, Count iterations)
{
    for (Count i = 0; i < iterations; ++i)
    {
        value.fetch_add(1);
    }
}
```

Ova imenovana funkcija menja jedan brojač. Parametar `value` je referenca, pa izmene ostaju u objektu pozivaoca. Svaka nit dobija zaseban element niza:

```
for (int id = 0; id < num_workers; ++id)
{
    threads.emplace_back(increment,
        ref(counters[id].value), iterations);
}
for (auto& thread : threads)
{
    thread.join();
}
```

Programski dodeljen ID je `id`, u opsegu $0, \ldots, p-1$ za p niti. Pomoću funkcije `ref` brojač se prosleđuje po referenci, a broj iteracija se kopira. Poziv metoda `join()` čeka završetak niti pre čitanja rezultata. Isti posao može se zapisati lambdom:

```
threads.emplace_back([&, id]
{
    increment(counters[id].value, iterations);
});
```

Vrednost identifikatora se kopira; niz brojača i broj iteracija koriste se po referenci. Oni moraju ostati živi do kraja svih niti. Lambda je alternativni zapis; priloženi program koristi imenovanu funkciju.

4.3.2 Raspored i poravnanje

```
using Count = unsigned long long;

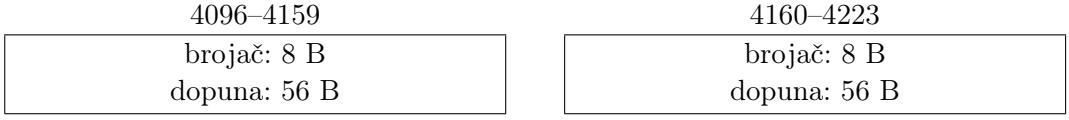
struct Counter
{
    atomic<Count> value{0};
};

struct alignas(PML_LINE_BYTES) PaddedCounter
{
    atomic<Count> value{0};
};
```

Svaki bajt memorije ima adresu. Objekat zauzima niz susednih bajtova, a njegova početna adresa je adresa prvog od njih. Poravnanje na A bajtova znači da početna adresa mora biti deljiva sa A, bez ostatka. Za poravnanje na 64 bajta, na primer, dozvoljeni počeci su 4096, 4160 i 4224. Početak na adresi 4104 ne zadovoljava to poravnanje.

Ključna reč `alignas` u C++-u zadaje zahtev za poravnanje. U zapisu `struct alignas(PML_LINE_BYTES) PaddedCounter` taj zahtev važi za svaki objekat tipa `PaddedCounter`. Konstanta `PML_LINE_BYTES` zadaje broj bajtova i u ovom programu podrazumevano ima vrednost 64. Zato ova deklaracija koristi poravnanje `alignas(64)`: kompajler obezbeđuje da svaki objekat tog tipa počne na adresi deljivoj sa 64.

Kompajler po potrebi dodaje dopunu, odnosno bajtove koji ne sadrže brojač, kako bi i sledeći element niza počeo na odgovarajućoj adresi. Za brojač od 8 B, u primeru sa strukturom od 64 B, raspored izgleda ovako:



Slika 4.1: Primer dva poravnata objekta, po jedan u svakoj liniji keša od 64 B. Dopuna obezbeđuje razmak između brojača.

Veličina koju vraća izraz `sizeof(PaddedCounter)` obuhvata i dopunu. Susedni elementi vektora razmaknuti su za toliko bajtova. Kada zadato poravnanje odgovara veličini linije keša, brojači iz ovog rasporeda nalaze se u različitim linijama. Program ispisuje veličine struktura i stvarne adrese, pa se raspored može proveriti na radnom računaru.

Proveriti veličinu linije na mašini pre tumačenja:

```
g++ -O3 -Wall -Wextra -pthread \
    src/arch/false_sharing.cpp -o src/false_sharing
./src/false_sharing --check
./src/false_sharing 2 10000 compact-first counters_2.csv
```

Za utvrđenu liniju od 128 B, na primer, u komandu kompajliranja dodati `-DPML_LINE_BYTES=128`. Zadati vrednost dobijenu za stvarnu mašinu. Skripta za prikupljanje podataka čita L1 podatke iz `sysfs` i beleži odakle potiče veličina; ako ih nema, koristi jasno označenu pretpostavku od 64 B. Ručno utvrđena vrednost zadaje se opcijom `-line-bytes`. Ako je atomska operacija realizovana unutrašnjim zaključavanjem, to uključiti u objašnjenje vremena.

Za svaku stvarnu adresu A_t proveriti broj linije $\lfloor A_t/L \rfloor$. Adresa prvog kompaktnog brojača ne mora biti poravnata: zato ne oduzimati A_0 pre deljenja. Za ilustrativne adrese 4128, 4136, 4144, 4152, 4160 i 4168 i $L = 64$ B grupisati ID-eve po liniji. Zatim isto uraditi sa adresama iz sopstvenog ispisa. Podatak `shared_lines` pokazuje broj linija u kojima su brojači bar dve niti.

Ako je taj broj nula za kompaktni raspored, taj raspored nije stvorio uslov za lažno deljenje. Za jednu nit to je očekivano.

Nacrtati dva privatna keša i jednu liniju sa dva brojača. Pratiti upise nit 0, nit 1, nit 0: kada prelazi pravo na upis? Uporediti sa brojačima u različitim linijama. Prelaz ne mora biti čitanje iz DRAM-a.

4.4 Prikupljanje cele serije i grafici

Za lokalnost obavezna serija koristi $N = 128, 512, 2048$. Za brojače ukupan rad je fiksiran na $K = 4000000$ ažuriranja. Za $p = 1, 2, 4$ svaka nit ima $n = K/p$ iteracija, ako su ti brojevi niti dostupni. Oba rasporeda pri istom p imaju iste operacije i isti rad. Program proverava *svaki* brojač, čiji rezultat mora biti n. Brojači se vraćaju na nulu pre svakog poziva, van merenja.

Dva rasporeda su alocirana jednom, pa njihove adrese ostaju iste tokom pet ponavljanja. Vreme obuhvata kreiranje niti, sva ažuriranja i čekanje `join`; alokacija, nuliranje, ispis adresa i provera su van intervala. Za svaki raspored program `false_sharing` najpre izvrši jedan pripremni poziv, čije vreme ne čuva. Zatim meri pet narednih izvršavanja svakog rasporeda i smenjuje njihov redosled. Niti se ne vezuju automatski za jezgra. Mali broj iteracija može više meriti pokretanje niti nego saobraćaj linija keša.

```
python3 scripts/run_experiments.py 4 --out results/lab4
python3 scripts/plot_results.py results/lab4
```

Skripta ponovo kompajlira oba programa, pokreće provere i bira 1, 2 i 4 niti u granicama dostupnih CPU-a. Za najviše dve niti dodati `-max-workers 2`. Puna serija sa četiri niti ima šest konfiguracija, po deset redova: dve varijante puta pet ponavljanja. Vremena pripremnih izvršavanja ne upisuju se u CSV. Skripta ne prepisuje postojeći direktorijum.

U direktorijumu rezultata su:

- komande kompajliranja u fajlu `build.txt` i podaci o računaru, okruženju i granicama merenja u fajlu `machine.json`; dopisati podatke o virtuelizaciji i drugom opterećenju;
- rezultati provera u fajlovima `checks.txt` i `checks.err`, sirova vremena u `run_*.csv`, konzolni izveštaji u `run_*.txt` i poruke o greškama u `run_*.err`;
- tabela `summary.csv` sa srednjom vrednošću, minimumom, maksimumom i odnosima vremena, kao i grafici `locality.png` i `false_sharing.png`.

Na graficima je srednje vreme, a opseg označava najmanje i najveće od pet vremena, bez tvrdnje o intervalu pouzdanosti. Za crtanje je potreban Matplotlib, kao u prve tri vežbe.

4.4.1 Kako tumačiti metrike

Od pet vremena do jedne srednje vrednosti

Za svaku varijantu i istu konfiguraciju imate pet vremena $T_1, \ldots, T_5$, u sekundama. Skripta računa

$$\overline{T} = \frac{T_1 + T_2 + T_3 + T_4 + T_5}{5}.$$

U `summary.csv` to je `mean_s`; `min_s` i `max_s` su najkraće i najduže od tih pet vremena. Na grafiku je tačka srednja vrednost, a krajevi uspravne linije su minimum i maksimum. Veći raspon pokazuje da su se trajanja više menjala između ponavljanja. Za malu razliku srednjih vrednosti najpre pogledati taj raspon i sirove rezultate, pa proveriti da li se razlika ponavlja u novoj seriji.

Porediti redove sa istim N za matricu, odnosno istim brojem niti p i istim ukupnim brojem ažuriranja K za brojače. Prvo usrednjiti vremena svake varijante, pa podeliti te srednje vrednosti.

Odnos vremena: šta je u brojiocu, a šta u imeniocu

Za matricu skripta deli vreme prolaza po kolonama vremenom prolaza po redovima. Za brojače deli vreme kompaktnog rasporeda vremenom poravnatog, pri istom p:

$$R_{\text{locality}} = \frac{\overline{T}_{\text{column}}}{\overline{T}_{\text{row}}}, \qquad R_{\text{layout},p} = \frac{\overline{T}_{\text{compact},p}}{\overline{T}_{\text{padded},p}}.$$

Imena u `summary.csv` doslovno prate ovo deljenje: `column_over_row` i `compact_over_padded`. Oba odnosa su bez jedinice: sekunde u brojiocu i imeniocu se skraćuju. Ista vrednost odnosa stoji u oba reda te konfiguracije, jer opisuje poređenje para varijanti.

Odnos	Matrica	Brojači, isti p
$R > 1$	Kraće traje prolaz po redovima.	Kraće traje poravnati raspored.
$R = 1$	Srednja vremena su jednaka.	Srednja vremena su jednaka.
$R < 1$	Kraće traje prolaz po kolonama.	Kraće traje kompaktni raspored.

Izmereni primer: prolazi kroz matricu

U tabeli su prikazani rezultati dva prolaza izmereni na procesoru AMD Ryzen 7 7700X.

U oba prolaza sabiraju se isti elementi, sa suprotnim redosledom petlji. Ulaz je matrica 2048×2048 sa četvorobajtnim elementima `int`, ukupno 16 MiB, popunjena kao `a[i] = i % 17 - 8`. Posle jednog pripremnog poziva svake varijante izmereno je po pet izvršavanja, uz smenjivanje redosleda. Meri se samo poziv funkcije za sabiranje; priprema, provere i ispis su izvan tog intervala. Prolaz po redovima je sekvencijalna referentna varijanta.

Prolaz	Srednje [ms]	Najkraće [ms]	Najduže [ms]
Po redovima	0.398758	0.389386	0.421046
Po kolonama	16.891316	16.758334	17.201262

Iz nezaokruženih srednjih vrednosti dobijamo

$$R_{\text{locality}} \approx \frac{16.891316}{0.398758} \approx 42.36.$$

U ovoj seriji prolaz po kolonama traje približno 42.36 puta duže. Oba prolaza izvršava jedna nit; upoređujemo redosled pristupa i način na koji se izvršavaju odgovarajuće petlje.



Slika 4.2: Izmerena srednja vremena za istu matricu na Ryzen 7 7700X. Dužine traka su srazmerne vremenima; kraća traka znači kraće izvršavanje.

Izmereni primer: raspored brojača

Sledeći rezultati dobijeni su priloženim programom za brojače na procesoru AMD Ryzen 7 7700X, sa linijom keša od 64 B. U ovom primeru ukupan rad je $K = 20000000$ atomskih ažuriranja: za jednu nit to je 20 miliona, za dve po 10 miliona, a za četiri po 5 miliona ažuriranja po niti. Oba rasporeda koriste isti poziv `fetch_add(1)`. Kompaktna struktura zauzima 8 B, a poravnata 64 B; provera je za oba rasporeda pokazala da su atomske operacije realizovane bez unutrašnjeg zaključavanja biblioteke.

Svaki proces ograničen je na skup od p logičkih CPU-a koji pripadaju različitim fizičkim jezgrima. Raspoređivač operativnog sistema bira gde se pojedina nit izvršava unutar tog skupa;

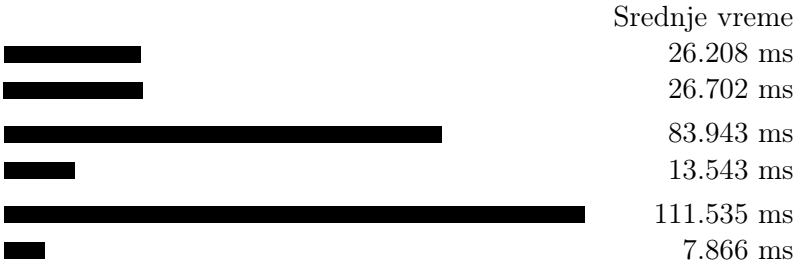
niti nisu pojedinačno vezane za jezgra. Posle jednog pripremnog izvršavanja svakog rasporeda izmereno je po pet izvršavanja, uz smenjivanje redosleda. Vreme uključuje kreiranje niti, ažuriranja i čekanje njihovog završetka. Alokacija, nuliranje, ispis i provera svakog brojača su izvan merenja.

Niti	Raspored	Srednje [ms]	Najkraće [ms]	Najduže [ms]
1	Kompaktni	26.208	25.378	27.092
1	Poravnati	26.702	26.349	26.881
2	Kompaktni	83.943	83.248	84.811
2	Poravnati	13.543	13.196	13.764
4	Kompaktni	111.535	108.502	116.222
4	Poravnati	7.866	6.805	11.133

Deljenjem nezaokruženih srednjih vremena kompaktnog i poravnatog rasporeda dobijamo $R_{\text{layout},1} \approx 0.98$, $R_{\text{layout},2} \approx 6.20$ i $R_{\text{layout},4} \approx 14.18$. Na primer, za četiri niti:

$$R_{\text{layout},4} \approx \frac{111.535}{7.866} \approx 14.18.$$

Ovaj odnos poredi dva rasporeda sa četiri niti i istim radom. Za tvrdnju o paralelnom ubrzanju potrebno je sekvencijalno referentno vreme za isti posao, kao u ranijim vežbama.



Slika 4.3: Isti ukupan broj atomskih ažuriranja pri različitom broju niti. Sve trake koriste istu razmeru. Opsezi pet izmerenih vremena dati su u tabeli.

Sa jednom niti vremena su bliska i njihovi opsezi se preklapaju. Nema druge niti koja bi tražila pravo na upis iste linije. Za dve i četiri niti provera adresa pokazala je jednu zajedničku liniju u kompaktnom rasporedu i nijednu u poravnatom. U izvršavanju sa četiri niti kompaktni brojači počinju na pomerajima 32, 40, 48 i 56 B unutar iste linije od 64 B. Poravnati brojači počinju na početcima četiri različite linije.

Promenom rasporeda svaka nit i dalje ažurira svoj brojač, ali više ne deli liniju sa brojačima drugih niti. Kod kompaktnog rasporeda ažuriranja različitih brojača traže pravo na upis iste linije, što povezuje veće vreme sa lažnim deljenjem. Dobijeni odnosi opisuju ovu seriju na navedenom procesoru. Za četiri niti poravnati raspored ima vidljiv raspon od 6.805 do 11.133 ms; tu promenljivost treba uzeti u obzir pri poređenju sa novom serijom. Samo vreme ne otkriva broj prenosa linije između keševa.

Koliko posla završimo u sekundi

Za matricu je broj obrađenih elemenata u jednom prolazu $M = N^2$. Ako element zauzima b bajtova, njihova ukupna veličina je $D = Mb$. Skripta računa:

$$q_{\text{elem}} = \frac{N^2}{\overline{T}} \quad [\text{elemenata/s}], \qquad q_{\text{useful}} = \frac{N^2 b}{\overline{T}} \quad [\text{B/s}].$$

To su `elements_per_s` i `useful_bytes_per_s`. Za isti posao kraće vreme daje veću brzinu obrade. U izmerenom primeru matrica zauzima 16 MiB, pa je brzina obrade korisnih podataka za prolaz po redovima 16 MiB/0.000398758 s ≈ 40125 MiB/s, a za kolone približno 947 MiB/s. Za pretvaranje kolone sa jedinicom B/s u MiB/s podeliti je sa 1024^2.

Ova brzina opisuje koliko korisnih bajtova matrice program obradi u sekundi. Podatak može stići iz keša, a jedna linija dovučena iz sledećeg nivoa može poslužiti za više pristupa. Linija koja je izbačena može se dovući ponovo; dovlačenje unapred i prevođenje adresa dodaju svoj saobraćaj. Zato se stvarna količina prenetih DRAM bajtova utvrđuje zasebnim brojačima tog prenosa. Odnos $N^2 b/\overline{T}$ sam daje brzinu obrade korisnih podataka.

Za brojače je `elements_per_s` broj ažuriranja u sekundi, $K/\overline{T}$. Mereni interval ovde obuhvata i kreiranje i čekanje niti. Kolona `useful_bytes_per_s` ostaje prazna za brojače: jedno atomsko ažuriranje samo po sebi ne određuje koliko je bajtova preneto između keševa.

Povezivanje rezultata sa arhitekturom procesora

U izmerenom primeru korak adresa je 4 B po redovima, a 8192 B po kolonama. Linija keša ovog procesora ima 64 B, pa u nju staje 16 elemenata koje prolaz po redovima koristi uzastopno. Matrica od 16 MiB veća je od L1 podataka od 32 KiB i L2 od 1 MiB tog jezgra, a manja od deljenog L3 od 32 MiB. Redosled pristupa utiče na ponovnu upotrebu linija, prevođenje adresa i mogućnosti vektorizacije petlje. Vremena pokazuju ukupnu razliku; doprinos svakog od tih mehanizama utvrđuje se dodatnim merenjem.

Za brojače povezati razliku vremena sa stvarnim adresama i brojem `shared_lines`. Izvršavanje sa jednom niti služi kao kontrola: tada nema druge niti koja bi preuzimala istu liniju.

4.5 Tumačenje i samostalna provera

Za jedan rezultat uporediti predviđanje sa merenjem i navesti adrese ili veličine koje podržavaju objašnjenje. Ako poravnanje nije pomoglo, proveriti linije, topologiju, preklapanje niti i trajanje petlje. Dopuna povećava memorijski otisak; poboljšanje nije garantovano.

Nastavnik zadaje jednu promenu: na primer $N = 1024$, obrnuti početni redosled prolaza ili udvostručen ukupan broj ažuriranja. Pre pokretanja zapisati predviđanje. Za novu veličinu matrice i isti ukupan rad može se koristiti:

```
python3 scripts/run_experiments.py 4 \
    --matrix-sizes 1024 --out results/lab4_n1024
python3 scripts/plot_results.py results/lab4_n1024
```

Nova serija sadrži i poređenje brojača. Jedna konfiguracija može se meriti i direktnom komandom uz novo ime CSV-a. Pokazati svoje funkcije, dodatnu matricu sa ručno izračunatim zbirom i izračunavanje linije stvarne adrese.

4.5.1 Neobavezno: brojači događaja

Posle uspešne provere programa, ova komanda ispisuje broj ciklusa, instrukcija i dostupnih keš promašaja, a program čuva svoja vremena u CSV:

```
perf stat -e cycles:u,instructions:u,cache-misses:u \
    -- ./src/locality 2048 row-first locality_perf.csv
```

Sufiks `:u` bira korisnički kod. Brojači obuhvataju ceo proces, uključujući pripremu i oba prolaza; `row-first` bira njihov redosled. Značenje `cache-misses` zavisi od CPU-a, pa ga ne tumačiti automatski kao L1 promašaje ili DRAM bajtove. Ako događaji nisu dostupni, sačuvati poruku alata i analizu zasnovati na vremenima i adresnom tragu.

Rezultat vežbe

- Pokazati dve svoje funkcije, dodatnu matricu sa ručno izračunatim zbirom i ručni adresni trag.
- Sačuvati ispravna merenja, podatke o mašini, `summary.csv` i dva grafika. Objasniti jednu razliku između predviđanja i merenja.
- Grupisati stvarne adrese brojača po linijama i objasniti jedan prelaz prava na upis. Predvideti i proveriti izmenjen parametar.